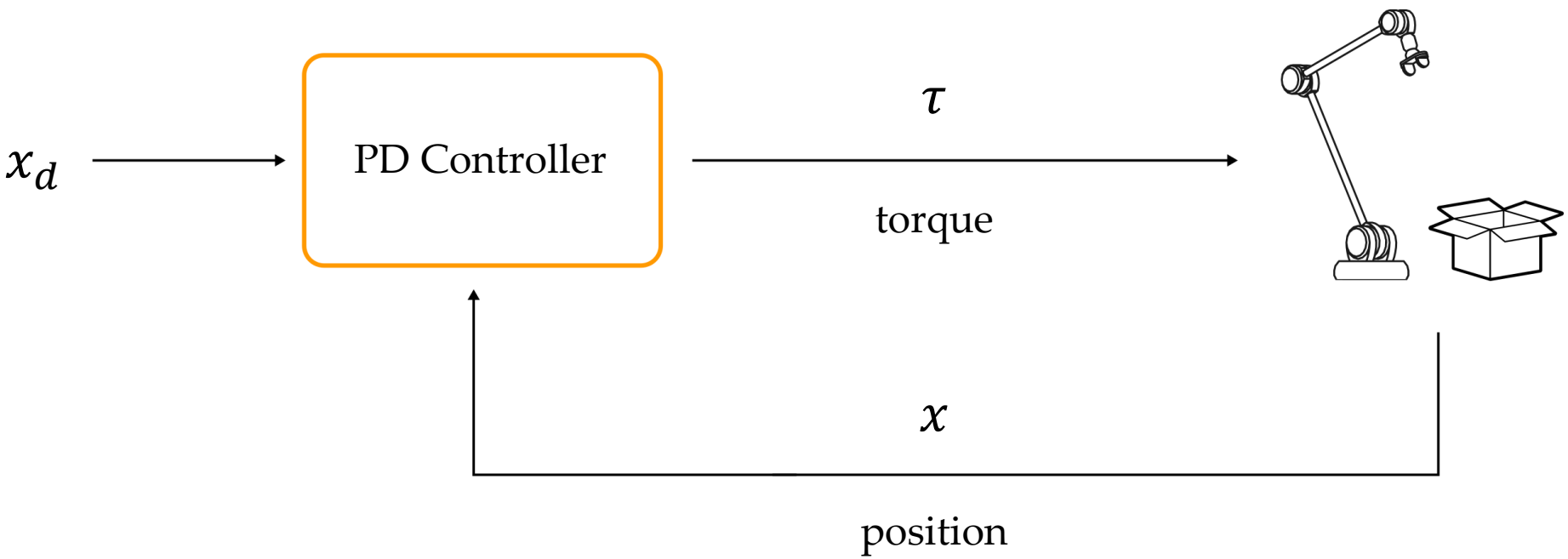
Imitation Learning (1)

Dylan Losey

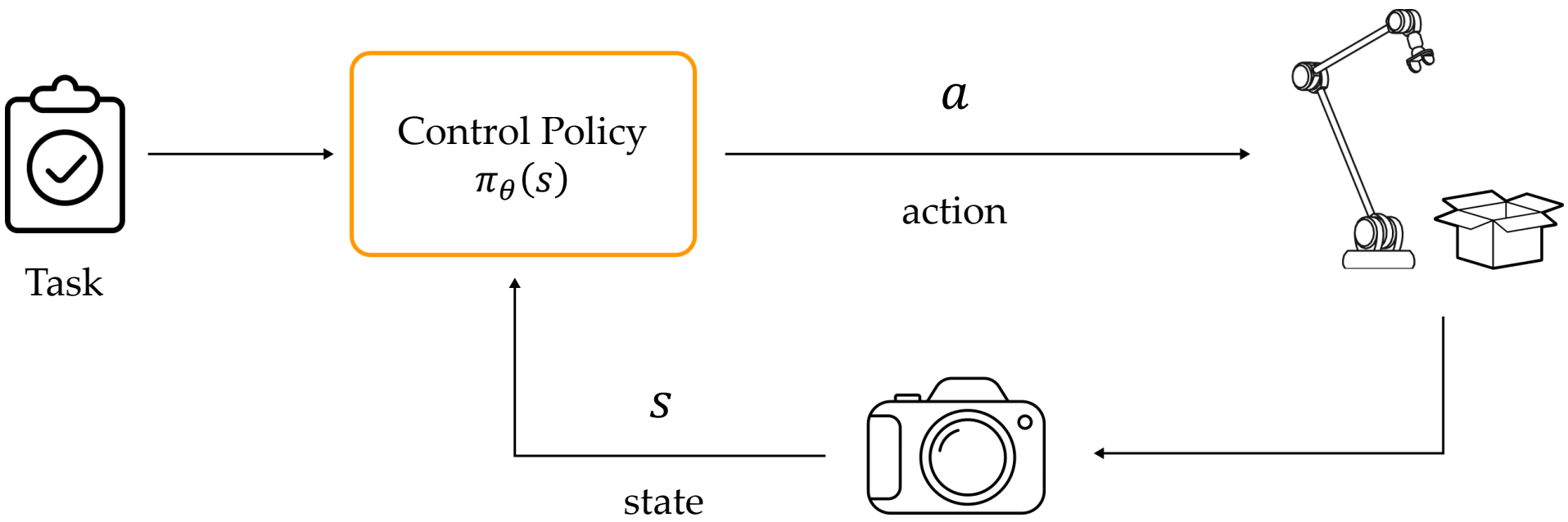

LAP 1 / 20
STANDINGS
1 TUM
2 UNIMORE
3 KINETIZ
4 TII RACING
5 POLIMOVE
6 CONSTRUCTOR
#A2RL

This Lecture

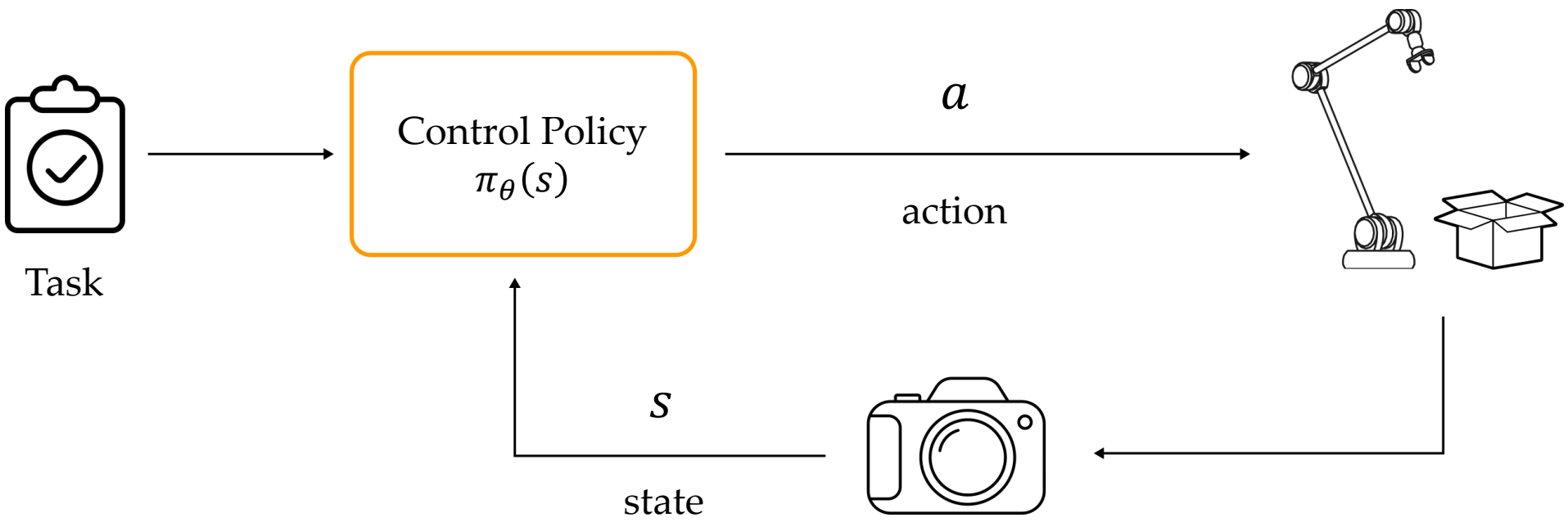
- What is robot learning?
- What is imitation learning?
- How can we leverage offline and online data?



Control. *The robot is given a desired position, and we design a stable controller to reach that position*



- [**Task**] robot is given a high-level objective, e.g., *pick up the box*
- [**State**] robot observes the environment (cameras, encoders), and combines the observed information into a state vector s
- [**Action**] robot chooses actions a to take. These actions could be joint torques τ, or the robot could output desired positions x_d



Learning. *The robot has a controller with parameters θ. The robot learns the right parameters to complete the task*

Robot Learning

The robot's controller is not fully specified by the designer. Instead, the designer provides a *function with parameters* (i.e., a **control policy**), and the robot autonomously tunes those parameters to *optimize its performance* (i.e., minimize its **loss function**)

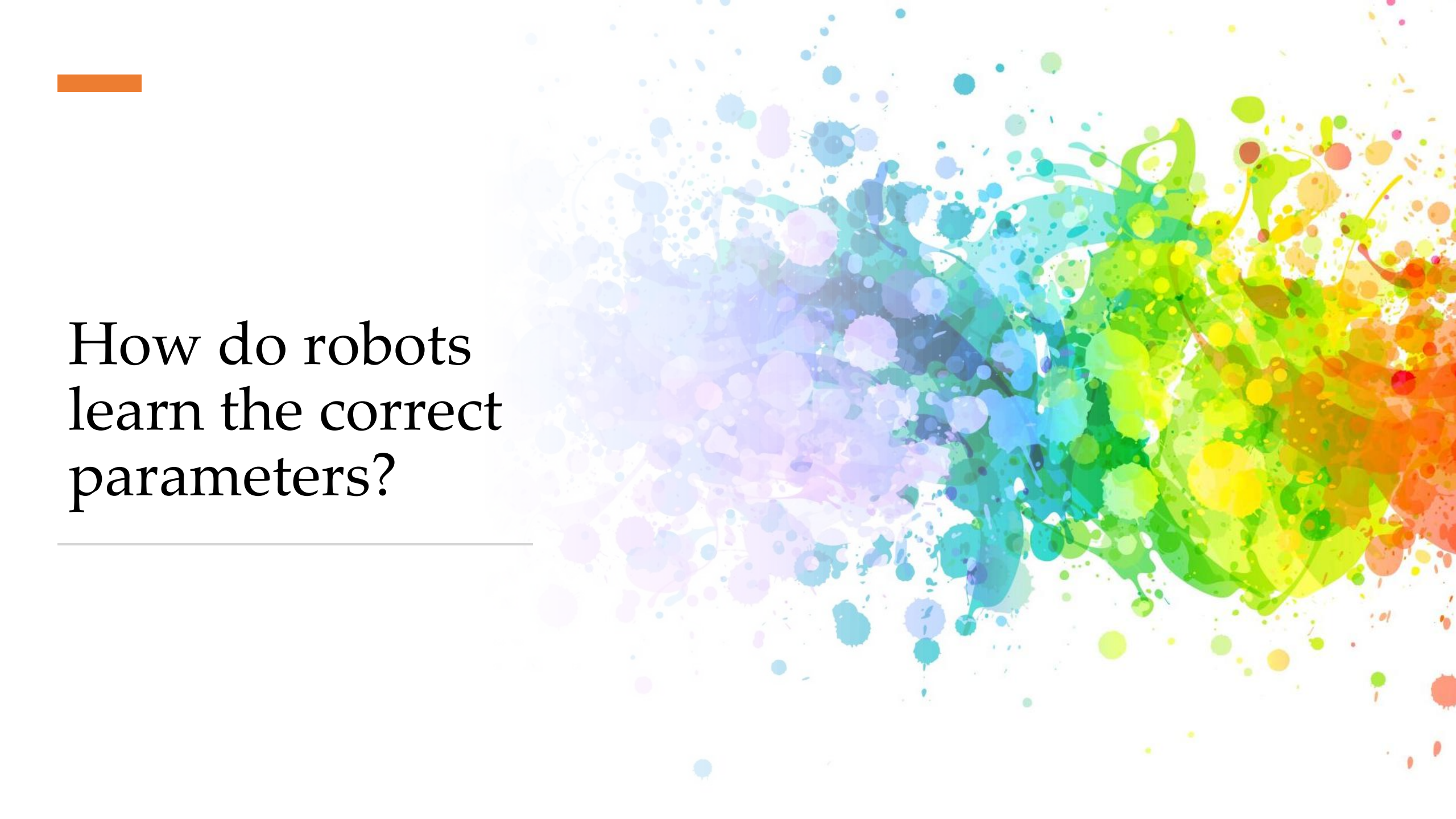
Robot Learning

The robot's controller is not fully specified by the designer. Instead, the designer provides a *function with parameters* (i.e., a **control policy**), and the robot autonomously tunes those parameters to *optimize its performance* (i.e., minimize its **loss function**)

$$a = \pi_\theta(s)$$

Policy maps what the robot observes (state s) to actions a and is parameterized by weights θ

How do robots learn the correct parameters?



Robot Learning

Control Policy. The robot's controller is not fully specified. Instead, the designer provides a function with parameters, and the robot tunes those parameters

Robot Learning

Control Policy. The robot's controller is not fully specified. Instead, the designer provides a function with parameters, and the robot tunes those parameters

Loss Function. The performance function we want the robot to optimize. The robot learns parameters to minimize this loss (*i.e., accuracy, error rate, cost…*)

Robot Learning

Control Policy. The robot's controller is not fully specified. Instead, the designer provides a function with parameters, and the robot tunes those parameters

Loss Function. The performance function we want the robot to optimize. The robot learns parameters to minimize this loss (*i.e., accuracy, error rate, cost…*)

We choose the control policy and loss function. We then collect training data, and the robot updates θ to minimize its loss across that training data

Demonstrations

Imagine we have collected a **dataset** of human behaviors:

$$D = \left(\left(s^1, a_h^1\right), \left(s^2, a_h^2\right) \ldots \left(s^N, a_h^N\right)\right)$$

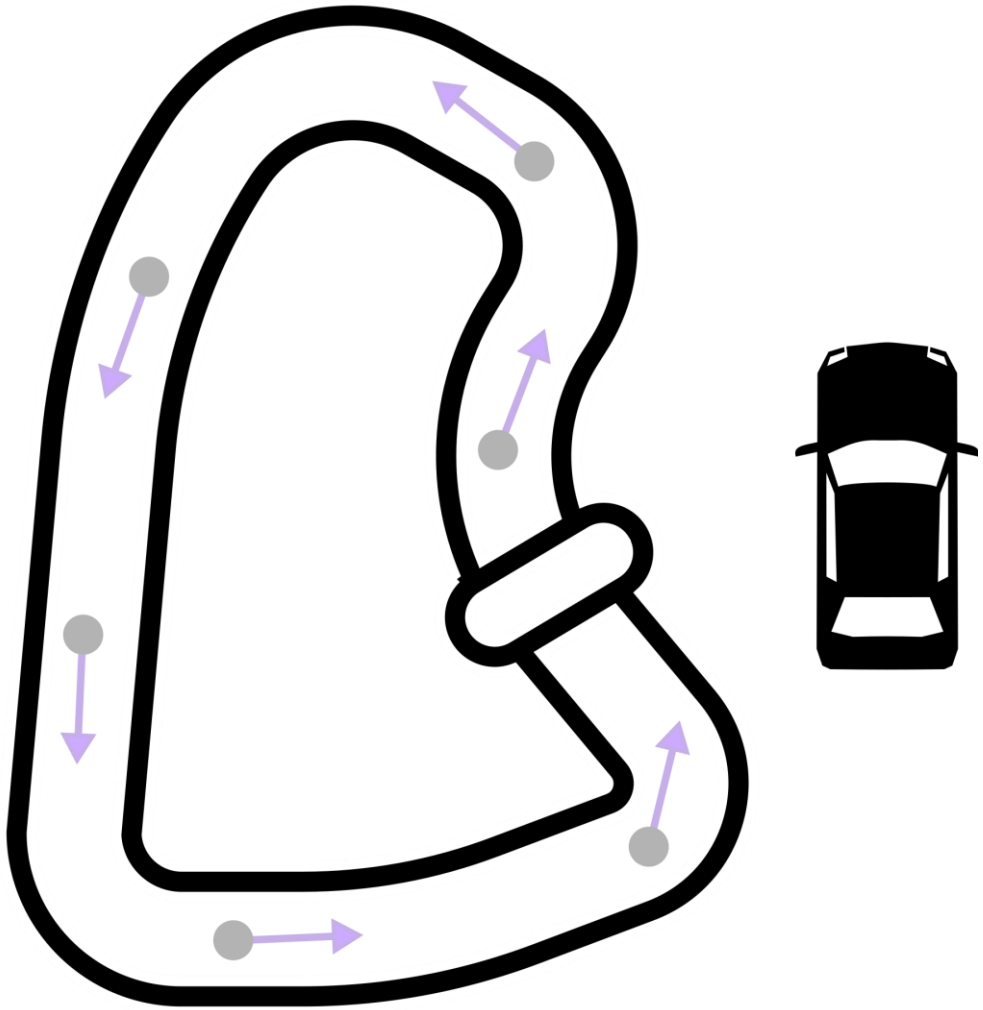
- Let s^t be the state at time t
- Let a_h^t be the human's *expert* action at time t

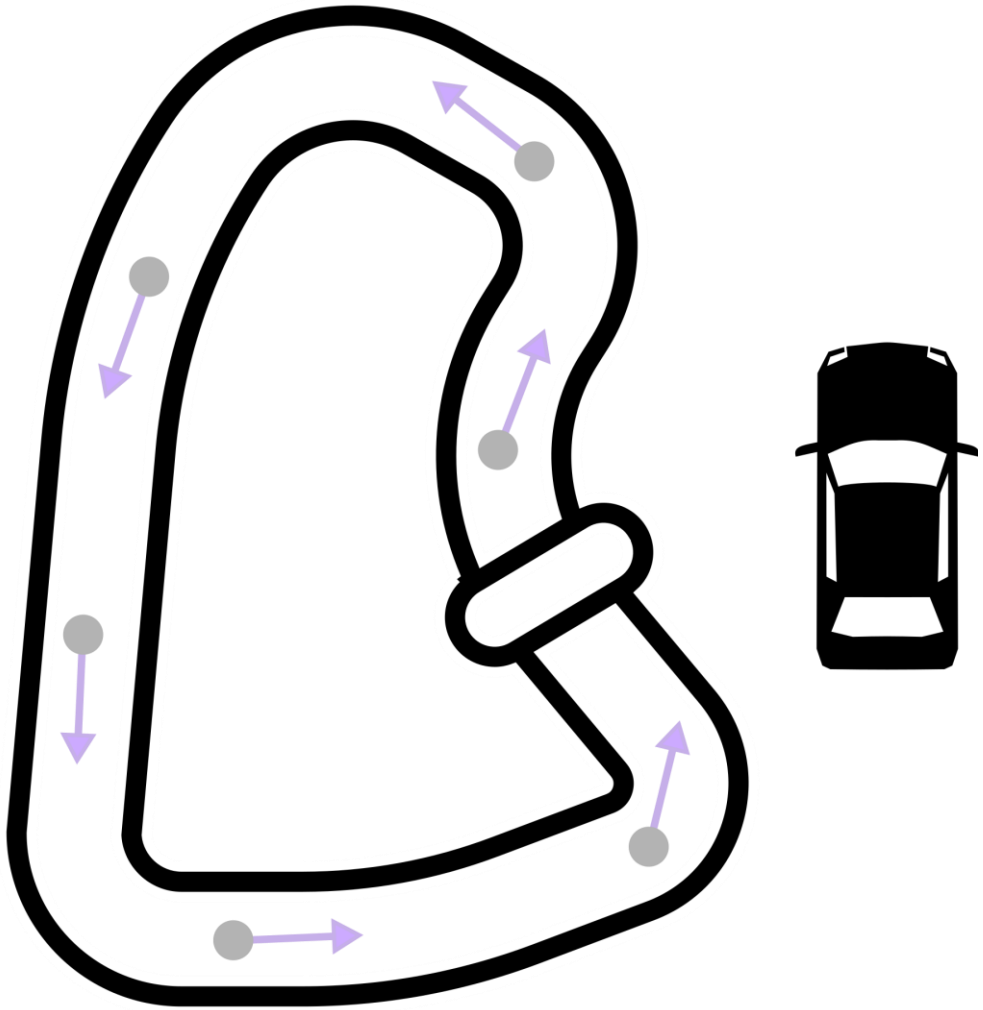
Demonstrations

Imagine we have collected a **dataset** of human behaviors:

$$D = \left(\left(s^1, a_h^1 \right), \left(s^2, a_h^2 \right) \ldots \left(s^N, a_h^N \right) \right)$$

Dataset D is a collection of N **state-action pairs**, where each state is labelled with the *expert* action at that state





Human **demonstrates** how to drive around this track.

State.

$$s = \begin{bmatrix} x \\ y \end{bmatrix}$$

Action.

$$a = \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix}$$

Dataset.

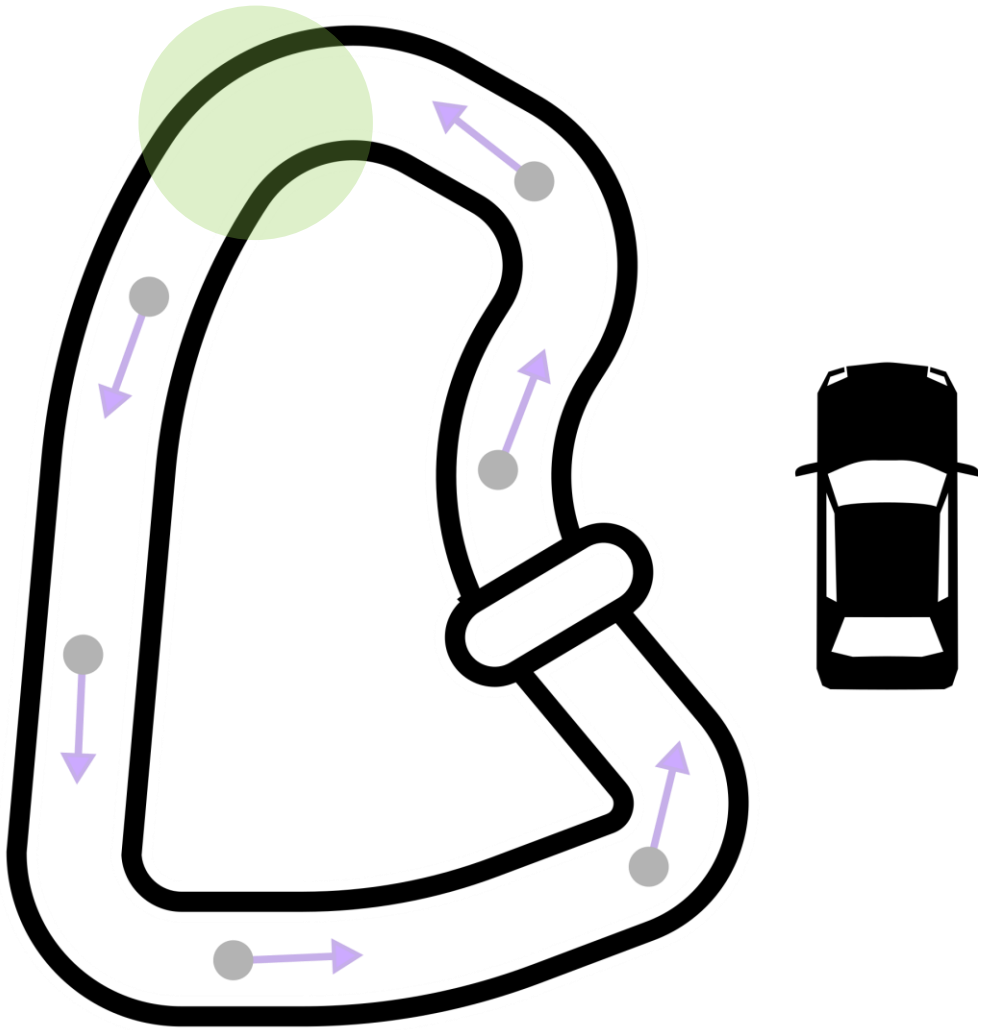
$$D = \left(\left(s^1, a_h^1 \right), \ldots, \left(s^N, a_h^N \right) \right)$$

Problem Setting

Given demonstrations of a task, the robot should:

1. **Match** the expert behavior at seen states

> *Why don't we just copy the human's actions exactly?*



What is the robot supposed to do **here**?

More generally, what do we do in states *without* expert labels?

Problem Setting

Given demonstrations of a task, the robot should:

1. **Match** the expert behavior at seen states
2. **Extrapolate** the expert behavior at new states

Problem Setting

Given demonstrations of a task, the robot should:

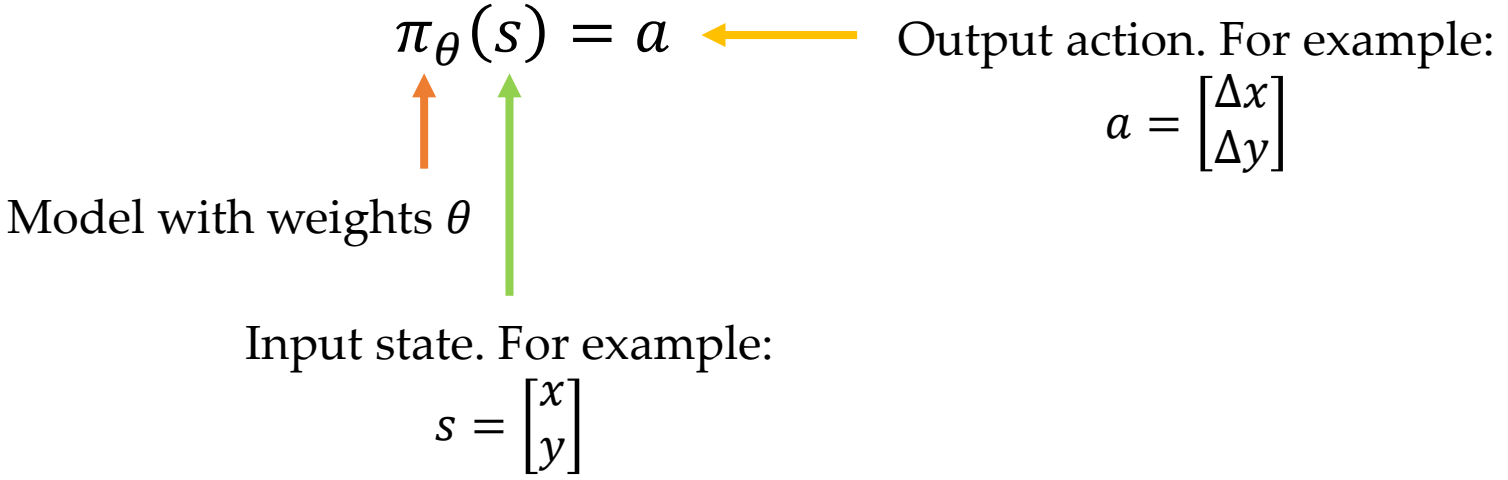
1. **Match** the expert behavior at seen states
2. **Extrapolate** the expert behavior at new states

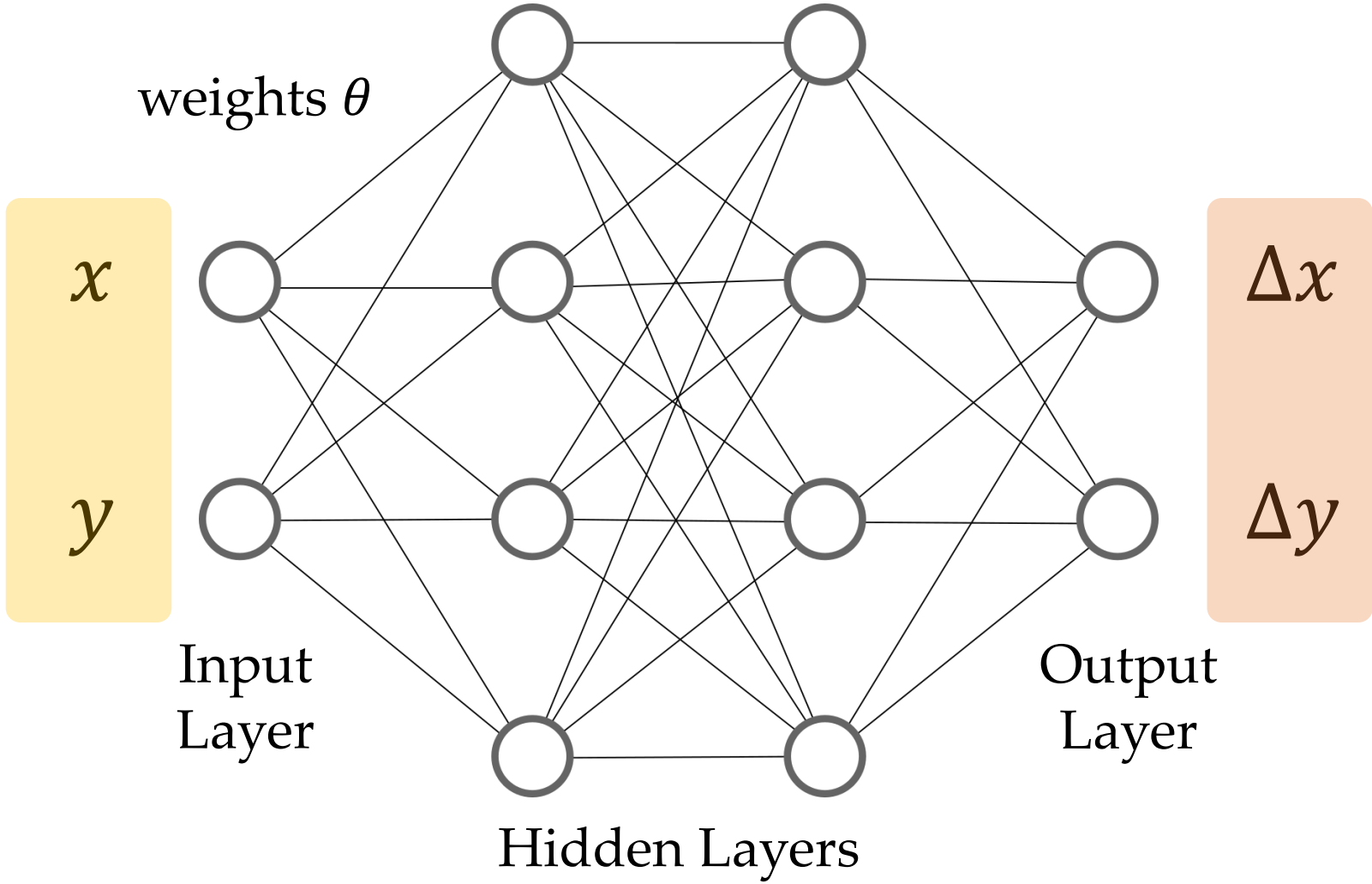
In behavior cloning, we are **given** expert dataset D our **objective** is to learn a robot policy $\pi(s)$ that matches and extrapolates the demonstrations.

Introducing our first imitation learning algorithm

Behavior Cloning

Let's instantiate the robot's policy as a model (neural network)




weights θ
x
y
Δx
Δy
Input Layer
Output Layer
Hidden Layers

Behavior Cloning

Loss Function.

We train the model to minimize the loss function:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{(s,a_h) \in D} \|a_h - \pi_\theta(s)\|^2$$

Loss depends on the model weights θ

Remember $\pi_\theta(s) = a$, so this is error between actual and predicted

Behavior Cloning

Loss Function.

We train the model to minimize the loss function:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{(s,a_h) \in D} \|a_h - \pi_\theta(s)\|^2$$

Want $\pi_\theta(s) = a_h$ across the entire dataset of expert state-action pairs

Behavior Cloning

1. **Collect** dataset of expert state-action pairs D
2. **Initialize** policy π_θ with random weights
3. **Train** the policy π_θ to learn weights that minimize:

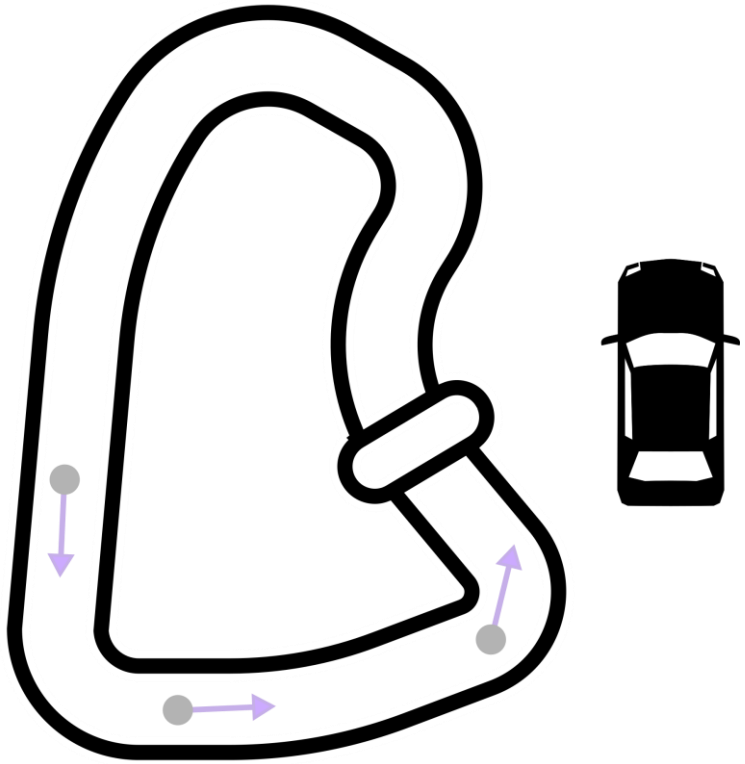
$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{(s,a_h) \in D} \|a_h - \pi_\theta(s)\|^2$$

Once done, control the robot using the learned policy.

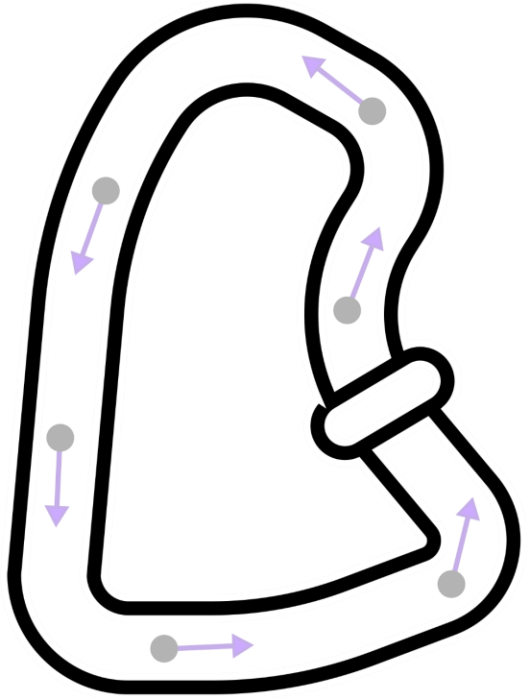
Out-of-Distribution

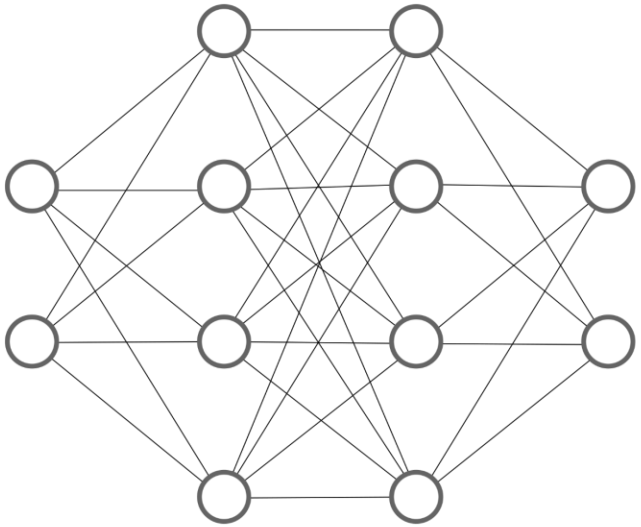
What happens when we encounter a state **unlike anything** in our dataset?

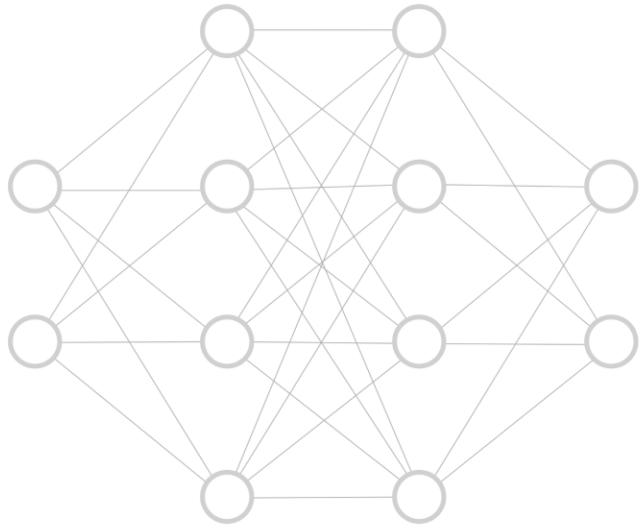
- **Example.** Large sections of the track without any human demonstrations.
- **Example.** Car makes small mistake and drives off track. We have no data on how to recover.



Offline







Online

How can we involve the human online and learn from new data?

Dataset Aggregation (DAgger)

- Given initial demonstrations D
- Train robot policy π to minimize $\mathcal{L}(\theta)$ over D

This is what we have already been doing

For timestep t in T:

$s \leftarrow$ current state

If [*condition*] is False:

$a \leftarrow \pi(s)$ take robot action

Else:

$a \leftarrow a_h$ take human action

$D \leftarrow D \cup (s, a_h)$ add to dataset

Every k timesteps:

Retrain policy π to minimize $\mathcal{L}(\theta)$ over D

Dataset Aggregation (DAgger)

- Given initial demonstrations D
- Train robot policy π to minimize $\mathcal{L}(\theta)$ over D

For timestep t in T:
    $s \leftarrow$ current state
    If [*condition*] is False:
        $a \leftarrow \pi(s)$ take robot action
    Else:
        $a \leftarrow a_h$ take human action
        $D \leftarrow D \cup (s, a_h)$ add to dataset
    Every k timesteps:
        Retrain policy π to minimize $\mathcal{L}(\theta)$ over D

Use the robot's policy to select actions unless **[condition]**

Dataset Aggregation (DAgger)

- Given initial demonstrations D
- Train robot policy π to minimize $\mathcal{L}(\theta)$ over D

For timestep t in T:

$s \leftarrow$ current state

If [*condition*] is False:

$a \leftarrow \pi(s)$ take robot action

Else:

$a \leftarrow a_h$ take human action

$D \leftarrow D \cup (s, a_h)$ add to dataset

Every k timesteps:

Retrain policy π to minimize $\mathcal{L}(\theta)$ over D

If **[condition]** *the human helps and we add their state-action pair to dataset*

Dataset Aggregation (DAgger)

- Given initial demonstrations D
- Train robot policy π to minimize $\mathcal{L}(\theta)$ over D

For timestep t in T:

$s \leftarrow$ current state

If [*condition*] is False:

$a \leftarrow \pi(s)$ take robot action

Else:

$a \leftarrow a_h$ take human action

$D \leftarrow D \cup (s, a_h)$ add to dataset

Every k timesteps:

Retrain policy π to minimize $\mathcal{L}(\theta)$ over D

Train the robot policy using old data + new data

Dataset Aggregation (DAgger)

- Given initial demonstrations D
- Train robot policy π to minimize $\mathcal{L}(\theta)$ over D

For timestep t in T:

$s \leftarrow$ current state

If [*condition*] is False:

$a \leftarrow \pi(s)$ take robot action

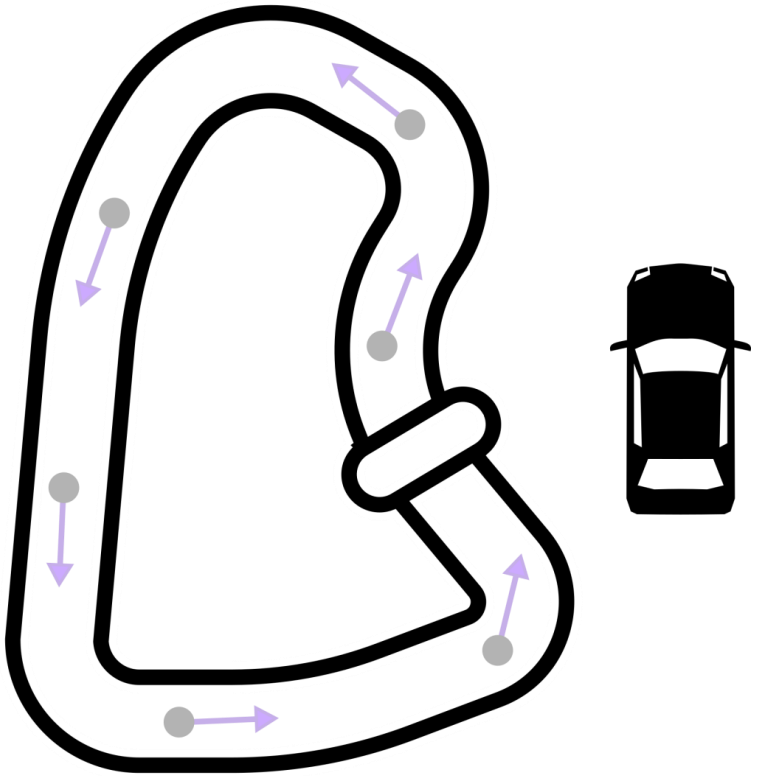
Else:

$a \leftarrow a_h$ take human action

$D \leftarrow D \cup (s, a_h)$ add to dataset

Every k timesteps:

Retrain policy π to minimize $\mathcal{L}(\theta)$ over D



Dataset Aggregation (DAgger)

- Given initial demonstrations D
- Train robot policy π to minimize $\mathcal{L}(\theta)$ over D

For timestep t in T:

$s \leftarrow$ current state

If [*condition*] is False:

$a \leftarrow \pi(s)$ take robot action

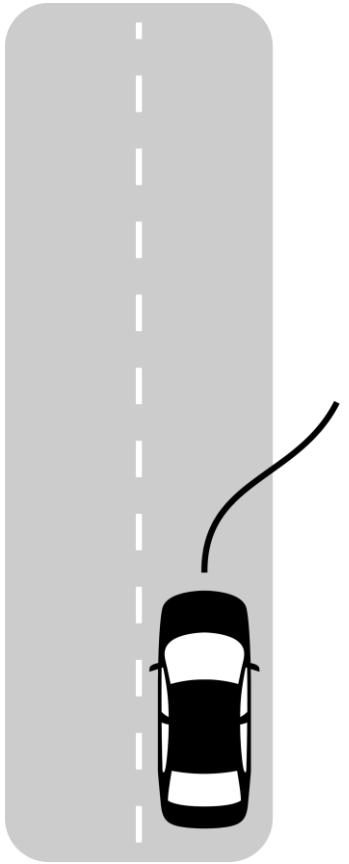
Else:

$a \leftarrow a_h$ take human action

$D \leftarrow D \cup (s, a_h)$ add to dataset

Every k timesteps:

Retrain policy π to minimize $\mathcal{L}(\theta)$ over D



Dataset Aggregation (DAgger)

- Given initial demonstrations D
- Train robot policy π to minimize $\mathcal{L}(\theta)$ over D

For timestep t in T:

$s \leftarrow$ current state

If [*condition*] is False:

$a \leftarrow \pi(s)$ take robot action

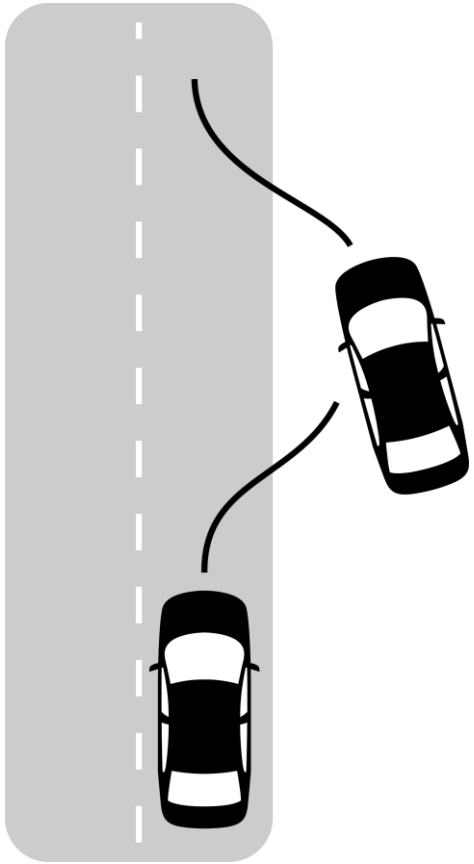
Else:

$a \leftarrow a_h$ take human action

$D \leftarrow D \cup (s, a_h)$ add to dataset

Every k timesteps:

Retrain policy π to minimize $\mathcal{L}(\theta)$ over D



Dataset Aggregation (DAgger)

- Given initial demonstrations D
- Train robot policy π to minimize $\mathcal{L}(\theta)$ over D

For timestep t in T:

$s \leftarrow$ current state

If [*condition*] is False:

$a \leftarrow \pi(s)$ take robot action

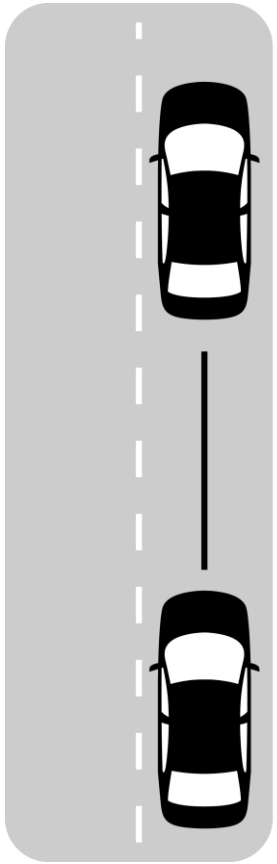
Else:

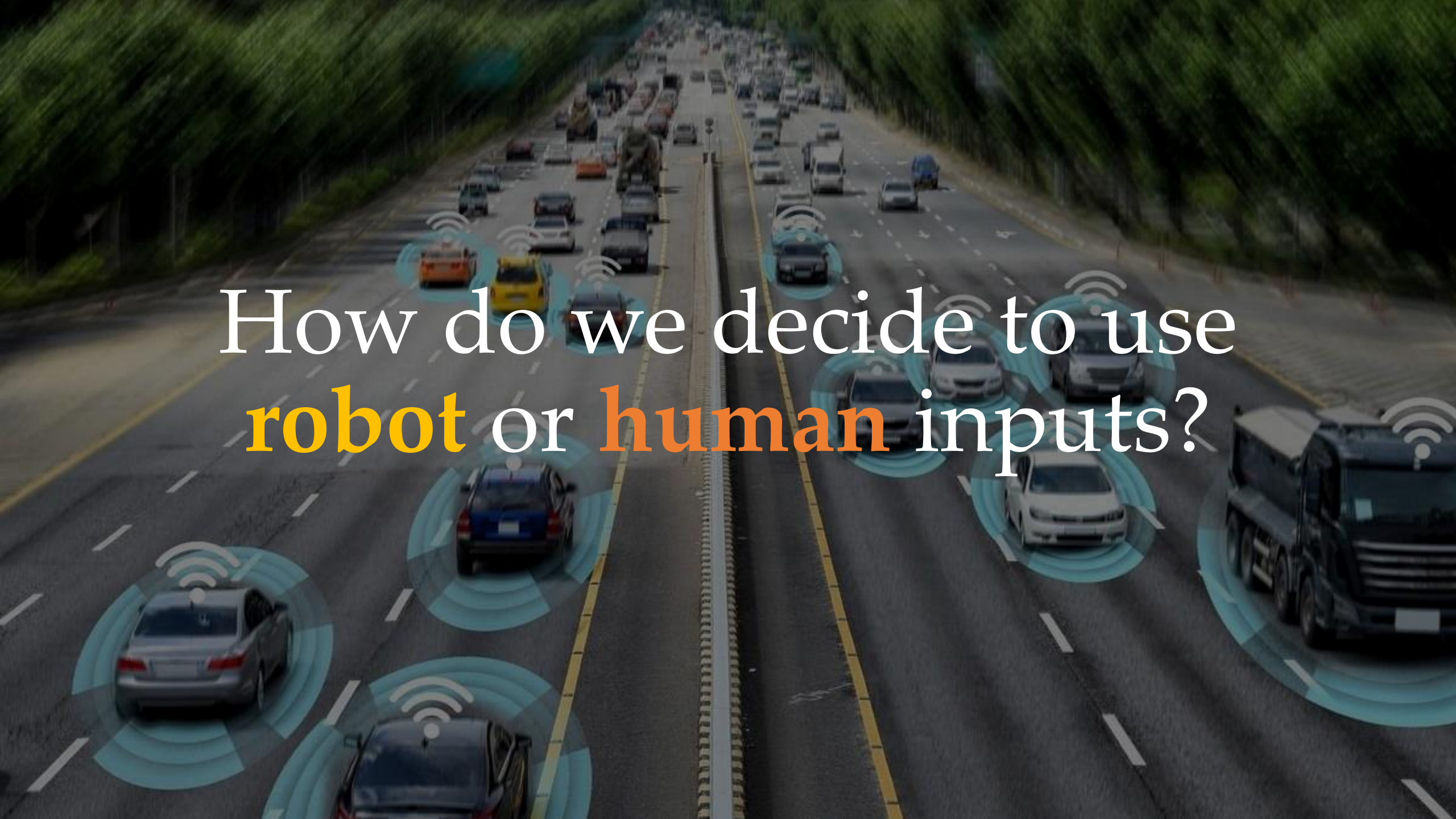
$a \leftarrow a_h$ take human action

$D \leftarrow D \cup (s, a_h)$ add to dataset

Every k timesteps:

Retrain policy π to minimize $\mathcal{L}(\theta)$ over D




How do we decide to use
robot or human inputs?

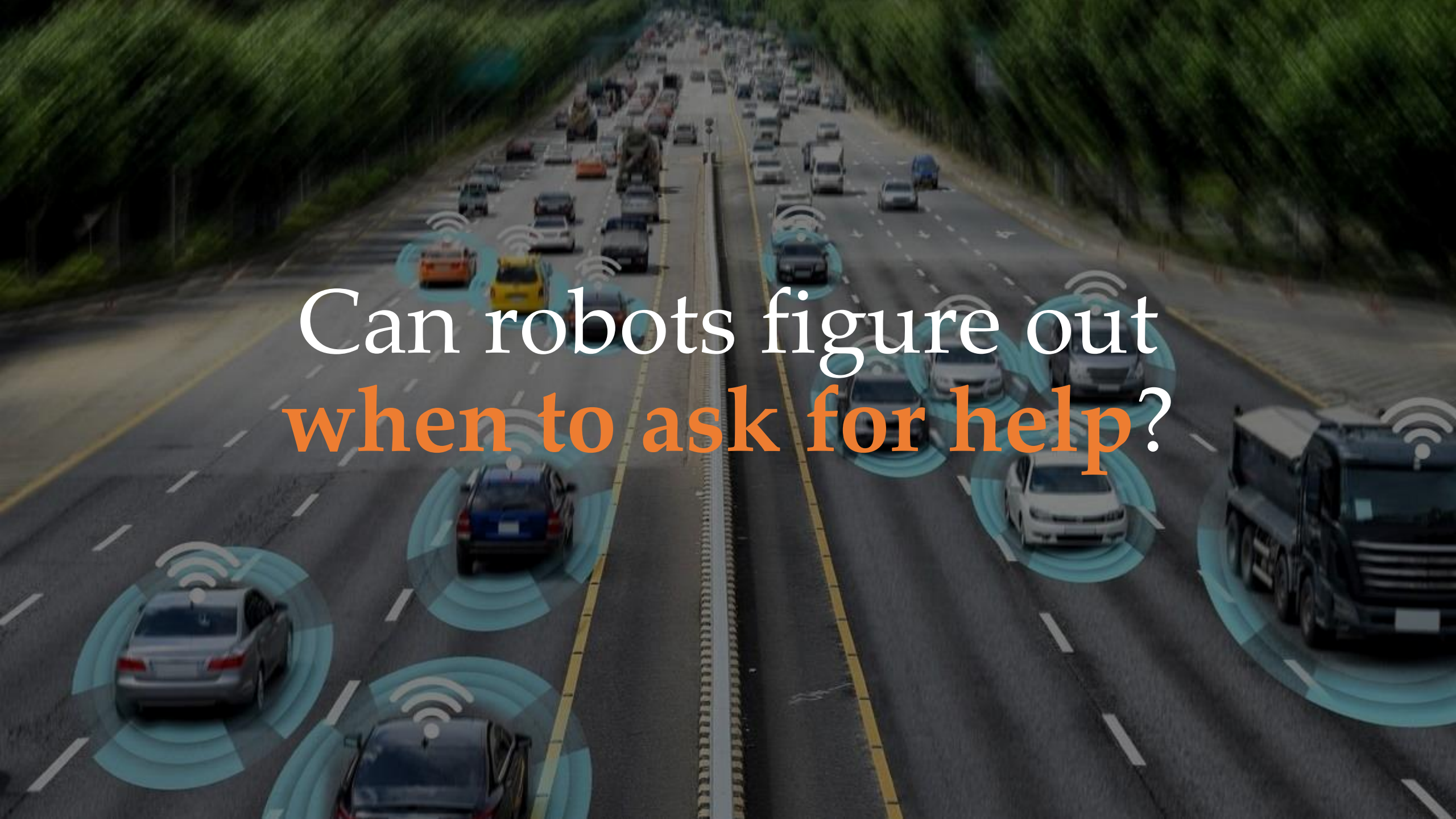
Human-Gated DAgger

In **human-gated DAgger** the human decides when to intervene.
Whenever the human intervenes they take over and guide the robot.

Human-Gated DAgger

In **human-gated DAgger** the human decides when to intervene.
Whenever the human intervenes they take over and guide the robot.

If [*no human input*]:
    $a \leftarrow \pi(s)$ take robot action
Else:
    $a \leftarrow a_h$ take human action
    $D \leftarrow D \cup (s, a_h)$ add to dataset


Can robots figure out
when to ask for help?

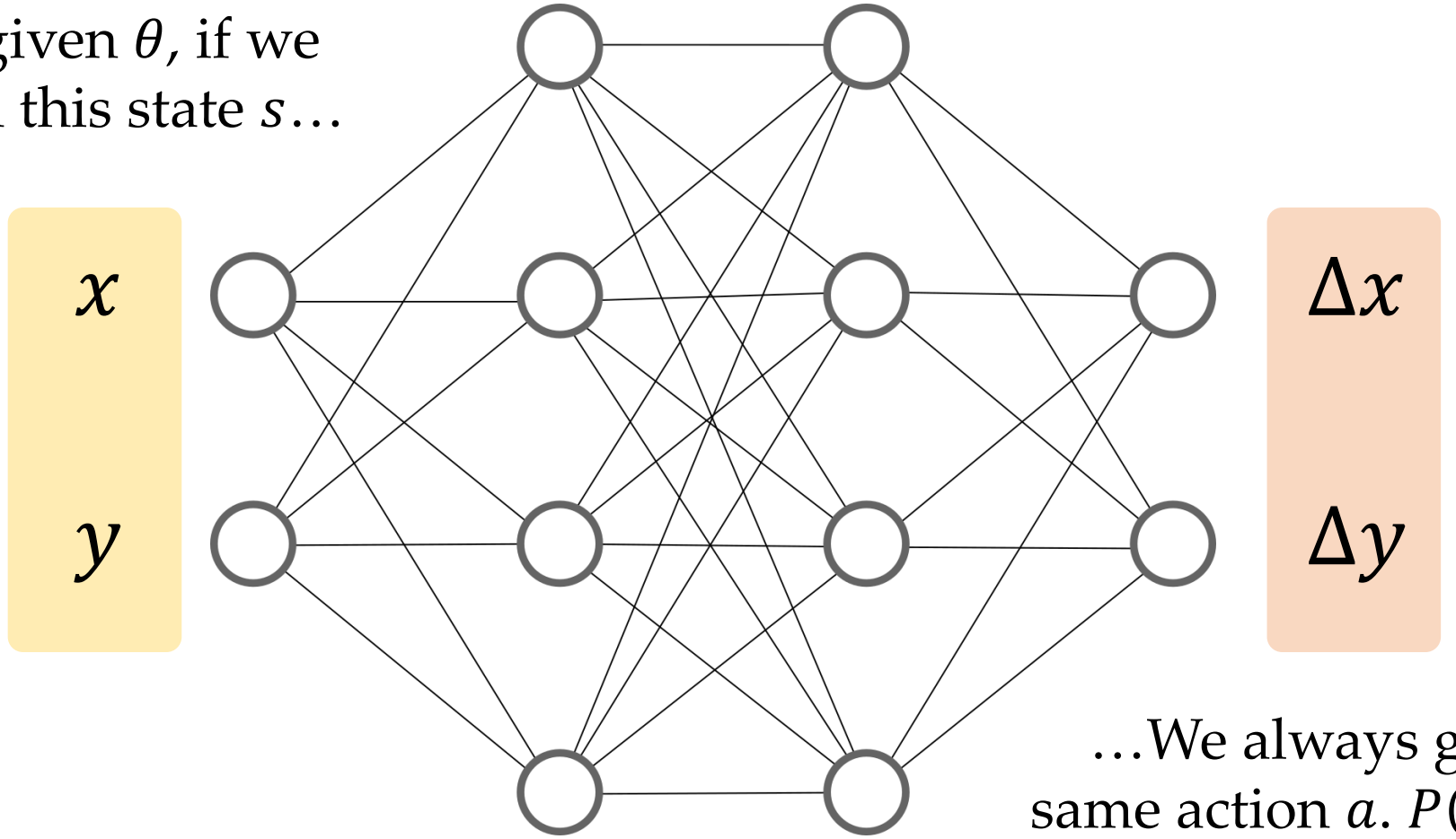
Robot-Gated DAgger

In **robot-gated DAgger** the robot must:

- Realize when it is uncertain
- Ask the human for help

This is really hard!
Remember the robot has policy $\pi_\theta(s) = a$

For given θ, if we put in this state s…



…We always get this same action a. $P(a|s) = 1$

Robot-Gated DAgger

In **robot-gated DAgger** the robot must:

- Realize when it is uncertain
- Ask the human for help

This is really hard!
Remember the robot has policy $\pi_\theta(s) = a$

Currently the robot has **no notion** of uncertainty or error.

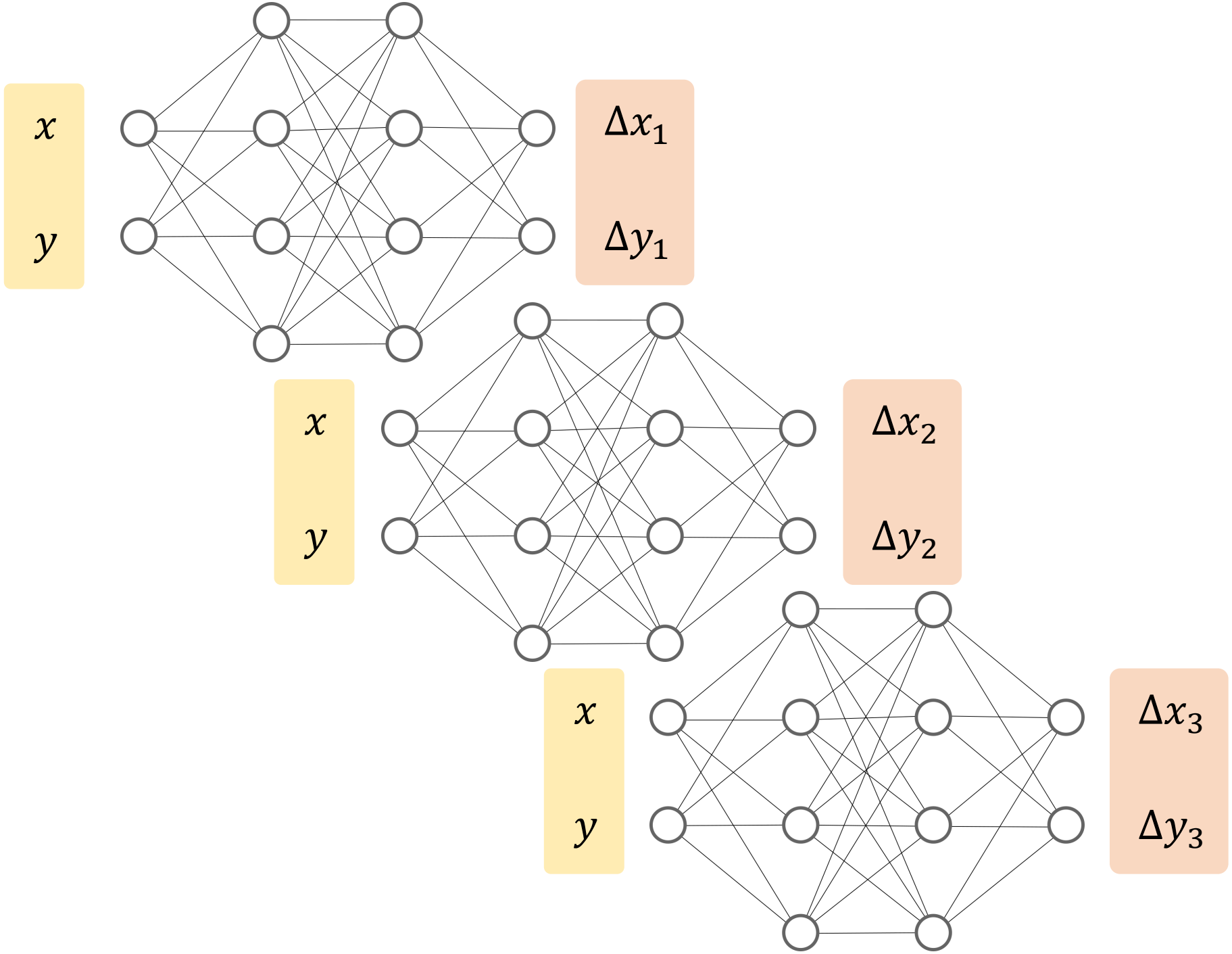
Robot-Gated DAgger

One example solution here is **EnsembleDAgger**

Train and update **multiple** robot policies to minimize $\mathcal{L}(\theta)$ over D

$$\pi_{\theta_1}, \pi_{\theta_2}, \ldots, \pi_{\theta_k}$$

- Each policy has the ***same*** structure and training procedure
- Because of initially randomized weights & stochasticity in training, each policy will converge to ***different*** θ


x
y
Δx_1
Δy_1
x
y
Δx_2
Δy_2
x
y
Δx_3
Δy_3

Robot-Gated DAgger

One example solution here is **EnsembleDAgger**

Train and update **multiple** robot policies to minimize $\mathcal{L}(\theta)$ over D

$$\pi_{\theta_1}, \pi_{\theta_2}, \ldots, \pi_{\theta_k}$$

If at state s the models **disagree** on what to do, ask for human help

$$\pi_{\theta_1}(s) = -2$$
$$\pi_{\theta_2}(s) = +5$$

Robot-Gated DAgger

One example solution here is **EnsembleDAgger**

$[a_1, \ldots a_k] \leftarrow [\pi_{\theta_1}, \ldots, \pi_{\theta_k}]$ evaluate all models

If stdev$(a_1, \ldots a_k) < \epsilon$:
    $a \leftarrow$ average of $[a_1, \ldots a_k]$
Else:
    $a \leftarrow a_h$ ask for human action
    $D \leftarrow D \cup (s, a_h)$ add to dataset

Takeaways

Control Policy. The robot's controller is not fully specified. Instead, the designer provides a function with parameters, and the robot tunes those parameters

Example: $a = \pi_\theta(s)$

Loss Function. The performance function we want the robot to optimize. The robot learns parameters to minimize this loss (*i.e., accuracy, error rate, cost…*)

Example: $\mathcal{L}(\theta) = \frac{1}{N}\sum_{(s,a_h)\in D}\|a_h - \pi_\theta(s)\|^2$

Training Data. In imitation learning we collect examples from humans (or other experts) and then update the policy parameters to match that expert behavior.

This Lecture

- What is robot learning?
- What is imitation learning?
- How can we leverage offline and online data?

Next Lecture

- Recent trends in imitation learning